

ELRS Ground Station

Benutzerhandbuch

Ein schlanker Ground-Control-Bildschirm für
ExpressLRS (ELRS) Modelle

Stand: August 2026

github.com/KresserSimon/ELRS_Telemetry_Groundcontrol

Inhalt

- 1. Einleitung
- 2. Installation
- 3. Erste Schritte
- 4. Verbindung zur Telemetrie herstellen
- 5. Die Benutzeroberfläche
- 6. Kartenoptionen (Satellitenbild, Sperrzonen, Rechtsklick-Menü, Home-Position)
- 7. Route/Wegpunkte planen, importieren und als INAV-Mission exportieren
- 8. Flugpfad-Aufzeichnung (Start/Pause/Export)
- 9. Fluglog (CSV-Aufzeichnung)
- 10. Plan-Modus
- 11. Die Menüs im Detail
- 12. Akkuwarnung: LiPo vs. Li-Ion
- 13. Als eigenständige .exe kompilieren
- 14. Anhang: Kommandozeilen-Referenz

1. Einleitung

ELRS Ground Station ist eine leichtgewichtige Alternative zu Mission Planner oder QGroundControl für Modelle mit ExpressLRS (ELRS). Die App zeigt live auf einer OpenStreetMap- oder Satellitenkarte, wo sich das Modell befindet, stellt alle wichtigen Telemetriedaten (GPS, Akku, Funkverbindung, Sensoren) in einem übersichtlichen Dashboard dar, warnt per Sprachausgabe bei niedrigem Akkustand und erlaubt es, Flugpfade aufzuzeichnen, Routen/Wegpunkte zu planen und als INAV-Mission zu exportieren.

Die App funktioniert mit zwei Telemetrie-Protokollen:

- MAVLink - ausgegeben von Flugsteuerungen wie ArduPilot, iNav oder Betaflight, die per CRSF-Telemetrie mit dem ELRS-Empfänger verbunden sind.
- CRSF (Crossfire) - das native Telemetrieprotokoll von ExpressLRS. CRSF wurde ursprünglich von TBS (Team BlackSheep) für deren Crossfire-Funksystem entwickelt; ExpressLRS verwendet bewusst dasselbe Frameformat, sodass die App auch mit echter TBS-Crossfire-Hardware funktioniert.

Beide Protokolle lassen sich sowohl über WiFi (UDP) als auch über eine direkte USB/seriell-Verbindung empfangen, umschaltbar zur Laufzeit ohne Neustart der App.

2. Installation

Voraussetzung ist Python 3.10 oder neuer. Im Projektordner:

```
cd elrs_ground_station
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

Die requirements.txt installiert PyQt6 (inkl. WebEngine für die Karte und WebChannel für die Interaktion mit der Karte), pymavlink, pytsx3 (Sprachausgabe) und pyserial (USB-Verbindungen). Unter Windows nutzt pytsx3 die eingebaute SAPI5-Sprachausgabe - es sind keine zusätzlichen Systempakete nötig.

Alternativ steht eine fertig kompilierte .exe zur Verfügung (siehe Abschnitt 13), die ohne Python-Installation läuft.

3. Erste Schritte

Zum unverbindlichen Ausprobieren ohne jegliche Hardware:

```
python main.py --demo
```

Der Demo-Modus simuliert einen Loiter-Kreisflug inklusive Akku-Entladung, Roll/Pitch-Bewegung, wechselnden Flugmodi und schwankender Funkverbindung, sodass sich alle Funktionen der App - einschließlich der Sprachwarnung bei niedrigem Akku - ohne echtes Modell testen lassen.

Wird die App ohne --demo gestartet, öffnet sich zunächst ein Popup zur Auswahl von Verbindung (WiFi/UDP oder USB) und Protokoll (MAVLink oder CRSF). Ein Klick auf Abbrechen übernimmt einfach die per Kommandozeile übergebenen bzw. die Standard-Einstellungen; zwei eigene Buttons im Popup starten stattdessen direkt den Demo- bzw. den Plan-Modus (siehe Abschnitt 10) - ohne jede Telemetriverbindung.

4. Verbindung zur Telemetrie herstellen

ELRS-Hardware spricht kein natives 'Telemetrie-über-WiFi' - das eingebaute WiFi eines ELRS-Moduls dient primär dem Flashen/Konfigurieren (Access Point ExpressLRS TX/ExpressLRS RX, Standardpasswort expresslrs). Um Telemetrie tatsächlich zu dieser App zu bekommen, gibt es drei Wege:

Weg 1: MAVLink über WiFi (empfohlen)

Voraussetzung ist eine Flugsteuerung mit MAVLink-Ausgabe, die per CRSF-Telemetrie mit dem ELRS-Empfänger verbunden ist. Der serielle MAVLink-Strom der Flugsteuerung muss per WiFi-Bridge (z. B. ein ESP32/ESP8266 mit MAVESP8266-Firmware) auf UDP-Port 14550 gebracht werden.

```
python main.py --protocol mavlink --host 0.0.0.0 --port 14550
```

Muss die Bridge stattdessen aktiv eine Verbindung zum PC aufbauen: --udp-mode connect --host <IP-der-Bridge>

Weg 2: Rohes CRSF/TBS-Crossfire über WiFi

Manche ELRS-'Backpack'-Bridges (ESP32-basiert) leiten den rohen CRSF-Bytestrom direkt per UDP weiter, ohne Umweg über MAVLink.

```
python main.py --protocol crsf --host 0.0.0.0 --port 14551
```

Dieser Modus deckt GPS (inkl. Geschwindigkeit), Akku (Spannung/Strom/Restkapazität/verbrauchte mAh), Link-Statistiken, Attitude (Roll/Pitch) sowie - falls gesendet - Vario, Baro-Höhe, RPM,

Temperatur und Zellspannungen ab.

Weg 3: USB/seriell

Flugsteuerung oder ELRS TX-Modul per USB-Kabel direkt an den PC anschließen. Verfügbare Ports zunächst auflisten:

```
python main.py --list-ports
python main.py --connection usb --protocol mavlink --serial-port COM5
python main.py --connection usb --protocol crsf --serial-port COM5 --baud 420000
```

Standard-Baudrate: 57600 für MAVLink, 420000 für CRSF (jeweils per --baud überschreibbar). Diese Verbindungsart ersetzt Weg 1/2, ist kein Zusatz dazu.

In allen WiFi-Fällen müssen PC und Bridge/Modul im selben Netzwerk sein. Alle drei Wege lassen sich auch nachträglich über Einstellungen -> Verbindung... ändern, ohne die App neu zu starten.

5. Die Benutzeroberfläche

5.1 Die Karte

Zeigt die Live-Position des Modells auf einer OpenStreetMap- oder Esri-Satellitenkarte (Karte -> Kartentyp), mit:

- Nachgezogenem, orangefarbenem Flugpfad seit Verbindungsbeginn.
- Einem Häuschen-Symbol an der Home-Position (dem ersten empfangenen GPS-Fix der laufenden Sitzung).
- Einem wählbaren Fahrzeugsymbol (Quadrocopter, Wing, Flugzeug), das sich mit der Kompassrichtung mitdreht.
- Auto-Center: die Karte folgt automatisch der aktuellen Position - abschaltbar per Menü oder per Klick auf den Lock-Button direkt auf der Karte (Google-Maps-Stil).
- Kartenausrichtung: ein zweiter fixer Kartenbutton schaltet zwischen Norden-oben und Drohnenrichtung-oben um. Im letzteren Modus dreht sich die gesamte Karte kontinuierlich mit dem aktuellen Kurs.
- Einer optionalen, per Klick oder Rechtsklick gezeichneten bzw. importierten Route (grün, gestrichelt, nummerierte Wegpunkte).
- Optional angezeigten No-Fly-Zones (rote Kreise/Polygone, siehe Abschnitt 6).

5.2 Das Dashboard

Die Leiste am unteren Fensterrand zeigt alle Telemetriedaten gruppiert an:

Gruppe	Felder
GPS	Breitengrad, Längengrad, Höhe, Satellitenanzahl
Status	Flugmodus
Link	RSSI, Link-Qualität (LQ), SNR, Sendeleistung
Akku	Spannung, Restkapazität, Min-Zellspannung, Strom, verbrauchte Kapazität (mAh)
Sensoren	Vario, Baro-Höhe, RPM, Temperatur
Long-Range	Geschwindigkeit, Entfernung/Peilung zur Home-Position, Flugzeit
Verbindung	Status-LED + Text (Verbunden/Getrennt)

Jedes einzelne Feld - nicht nur ganze Gruppen - lässt sich über Einstellungen -> Dashboard anpassen... ein- oder ausblenden. Die getroffene Auswahl wird als persönlicher Standard in `~/elrs_ground_station/dashboard_fields.json` gespeichert und bei jedem weiteren Start automatisch wieder geladen.

5.3 Verschieb- und größenveränderbare Karten-Overlays

Der künstliche Horizont, der Wegpunkt-Editor und die Tracking-Aufzeichnung erscheinen als eigene

Panels direkt auf der Karte - wie kleine Fenster, nicht als separate Dialoge. Jedes davon:

- lässt sich frei verschieben - einfach an einer nicht-interaktiven Stelle (Titelzeile) anklicken und ziehen,
- lässt sich an einer kleinen Anfassmarke in der unteren rechten Ecke mit der Maus größer oder kleiner ziehen, wie bei einem Fenster,
- lässt sich über Einstellungen -> Ansicht bzw. das jeweilige Menü komplett ein-/ausblenden.

5.4 Künstlicher Horizont

Zeigt Roll und Pitch als klassischer Fluglageanzeiger, gespeist aus MAVLink- oder CRSF-Attitude-Daten. Zusätzlich zum freien Verschieben/Skalieren per Maus bietet Einstellungen -> Ansicht feste Ecken-Presets (oben links/rechts, unten links/rechts) sowie feste Größenstufen (75%-200%).

6. Kartenoptionen

6.1 Kartentyp: OpenStreetMap oder Satellit

Karte -> Kartentyp schaltet zwischen OpenStreetMap (Standard) und Esri-Satellitenbildern um, ohne dass die Karte neu geladen werden muss.

6.2 No-Fly-Zones

Karte -> Sperrzonen laden... importiert Sperrzonen aus einer GeoJSON- oder CSV-Datei und zeigt sie als rote Kreise (CSV mit Radius) bzw. Polygone (GeoJSON Polygon/MultiPolygon) auf der Karte an.

Karte -> Sperrzonen anzeigen blendet sie ein/aus, ohne die geladenen Zonen zu verwerfen.

6.3 Rechtsklick-Menü

Ein Rechtsklick auf die Karte öffnet - unabhängig vom Wegpunkt-Modus - ein Kontextmenü mit folgenden Punkten:

- Wegpunkt / Startpunkt / Endpunkt - fügt an der angeklickten Stelle einen entsprechenden Punkt zur Route hinzu (siehe Abschnitt 7).
- Als Home setzen - teacht die Home-/Startposition (siehe 6.5) direkt an der angeklickten Stelle, ohne einen Dialog öffnen zu müssen.
- Ansicht (Untermenü) - Schnelzugriff auf Auto-Center, Kartenausrichtung, Wegpunkt-Editor anzeigen und Koordinaten anzeigen; jeder Eintrag schaltet exakt denselben Zustand wie das jeweilige Menü in der Menüleiste.

6.4 Koordinatenanzeige

Einstellungen -> Ansicht -> Koordinaten unter Mauszeiger anzeigen (oder das Ansicht-Untermenü im Rechtsklick-Menü) blendet ein kleines Overlay ein, das Lat/Lon direkt neben dem Mauszeiger anzeigt, während dieser sich über der Karte bewegt. Standardmäßig ausgeblendet.

6.5 Home-/Startposition

Einstellungen -> Home-Position... legt fest, wo die Karte beim nächsten Start zentriert ist - unabhängig von der live ermittelten Home-Position (immer der erste GPS-Fix der laufenden Sitzung), die für die Entfernungs-/Peilungsanzeige im Dashboard verwendet wird. Der Dialog bietet direkte Lat/Lon-Eingabe sowie einen Button 'Aktuelle Position verwenden', falls bereits ein GPS-Fix vorliegt. Alternativ lässt sich die Home-Position per Rechtsklick auf die Karte -> Als Home setzen direkt an der gewünschten Stelle teachen, ohne Koordinaten von Hand einzutippen. In beiden Fällen wird die Karte sofort zentriert bzw. gespeichert, ein Neustart der App ist nicht nötig.

7. Route/Wegpunkte planen, importieren und als INAV-Mission exportieren

Neben der live aufgezeichneten Flugspur (orange) kann eine unabhängige, geplante Route (grün, gestrichelt, mit nummerierten Wegpunkt-Markern) auf der Karte angezeigt werden - entweder von Hand gezeichnet (Route -> Wegpunkt-Modus, oder per Rechtsklick -> Wegpunkt/Startpunkt/Endpunkt) oder importiert aus einer Datei (Route -> Route importieren...).

7.1 Unterstützte Import-/Exportformate

Format	Beschreibung
.gpx	Liest bzw. schreibt Routenpunkte (<rte>); beim Import ersatzweise Track-Punkte oder einzelne Wegpunkte.
.mission (JSON)	Modernes INAV-Missionsformat (Schema-Version 1.0), siehe Abschnitt 7.3. Wird beim Import automatisch am führenden '{' erkannt.
.mission (XML)	Älteres 'MW XML'-Missionsformat von mwp/iNav Configurator/ezgui - <missionitem>-Elemente mit Breiten-/Längengrad in Dezimalgrad und Höhe in Metern.
.xml	Generischer Fallback: durchsucht beliebige XML-Strukturen nach Attributen oder Kindelementen mit erkennbaren Lat/Lon/Alt/Name-Bezeichnungen.
.csv	Erkennt Spalten wie lat/latitude, lon/longitude, optional alt und name (Groß-/Kleinschreibung egal); Export schreibt name/lat/lon/alt.

7.2 Der Wegpunkt-Editor

Route -> Wegpunkt-Editor anzeigen (auch unter Einstellungen -> Ansicht verfügbar) blendet ein Overlay direkt auf der Karte ein - keinen separaten Dialog. Oben zeigt es Wegpunktanzahl und Gesamtdistanz, darunter eine Tabelle mit je einer Zeile pro Wegpunkt: Lat/Lon (nur Anzeige), Höhe (relativ zur Home-Position, nicht Meereshöhe), Name sowie die INAV-Missionsparameter Aktion, Geschwindigkeit und P1-P3. Änderungen an der Tabelle wirken sofort auf die Route - es gibt keinen 'OK'-Button, der erst bestätigt werden muss.

Drei Buttons im Editor:

- Als INAV-Mission exportieren... / INAV-Mission importieren... - siehe Abschnitt 7.3.
- Gelände prüfen - fragt für jeden Wegpunkt die Geländehöhe ab (Open-Elevation-Dienst, benötigt Internetverbindung) und färbt die Zeile: rot = die geplante Flughöhe liegt unter der Geländehöhe (Kollision), gelb = weniger als 20 m Bodenfreiheit, grün = ausreichend Abstand.

7.3 INAV-Mission (.mission JSON)

Neben GPX/CSV lässt sich die Route auch im modernen INAV-Missionsformat (JSON, Schema-Version '1.0') exportieren und importieren - dem Format, das INAV Configurator selbst für Missionen verwendet. Unterstützte Aktionen:

Aktion	Bedeutung von P1/P2/P3
WAYPOINT	P1 = Wartezeit in Sekunden (0 = Durchflug ohne Halt).

Aktion	Bedeutung von P1/P2/P3
HOLD	P1 = Haltedauer in Sekunden.
RTH	Rückkehr zur Home-Position; Lat/Lon/Alt dürfen 0 sein.
SET_POI	Lat/Lon/Alt definieren das Kameraziel.
JUMP	P1 = Ziel-Wegpunktindex (1-basiert), P2 = Wiederholungsanzahl.
LAND	Automatische Landung an Lat/Lon.

Vor dem Export prüft die App die Route auf häufige Fehler (z. B. endet die Mission nicht mit RTH/HOLD/LAND, oder ein JUMP-Ziel liegt außerhalb der Route) und zeigt diese als Warnung, bevor die Datei geschrieben wird.

8. Flugpfad-Aufzeichnung (Start/Pause/Export)

Ein eigenes Overlay auf der Karte startet und pausiert die Aufzeichnung des geflogenen Pfads unabhängig von der Live-Anzeige: die Karte zeichnet den orangefarbenen Flugpfad immer nach, aber nur während die Aufzeichnung aktiv ist, landen Punkte im späteren Export. Nach dem Verbindungsaufbau (bzw. Start des Demo-Modus) beginnt die Aufzeichnung bewusst pausiert - ein Klick auf Start im Overlay setzt sie in Gang, ein erneuter Klick pausiert wieder; die Anzahl bisher aufgezeichneter Punkte wird laufend aktualisiert.

Exportieren... im Overlay fragt zunächst das gewünschte Format ab - GPX, KML oder CSV - und öffnet danach den Speicherort-Dialog für das gewählte Format.

Diese Aufzeichnung ist unabhängig vom Fluglog (Abschnitt 9): das Tracking speichert nur Positionspunkte für einen späteren Pfad-Export, das Fluglog schreibt alle Telemetriefelder kontinuierlich als Zeitreihe in eine CSV-Datei.

9. Fluglog (CSV-Aufzeichnung)

Das Fluglog zeichnet - unabhängig von der Flugpfad-Aufzeichnung aus Abschnitt 8 - sämtliche Telemetriedaten als Zeitreihe in einer CSV-Datei auf: Position, Höhe, Satelliten, GPS-Fix, Kompassrichtung, Roll/Pitch, Flugmodus, Akku (Spannung/Restkapazität/Strom/verbrauchte Kapazität/Zellspannungen), Funkverbindung (RSSI/LQ/SNR/Sendeleistung), Sensoren (Vario/Baro-Höhe/RPM/Temperatur), Geschwindigkeit und Verbindungsstatus.

Über Fluglog -> Log-Einstellungen... lässt sich sowohl die Menge der aufgezeichneten Spalten als auch das Aufzeichnungsintervall (0,1 bis 60 Sekunden) frei wählen. Nach Klick auf Fluglog -> Logging aktiv wird ein Speicherort abgefragt; ab dann schreibt die App laufend eine Zeile pro Intervall, bis das Häkchen wieder entfernt wird. Änderungen an den Log-Einstellungen wirken sich erst auf die nächste gestartete Aufzeichnung aus, um die bereits laufende CSV-Datei nicht durch einen inkonsistenten Spaltenkopf zu beschädigen.

10. Plan-Modus

Der Plan-Modus erlaubt es, Routen zu planen und Wegpunkte zu setzen, ohne dass irgendeine Telemetrie-Verbindung - echt oder simuliert - läuft. Aktivierbar über Simulation -> Plan-Modus oder direkt im Verbindungs-Popup beim Programmstart. Beim Aktivieren wird die Auto-Center-Sperre automatisch gelöst, da der Sinn des Plan-Modus gerade darin besteht, frei über die Karte zu navigieren, ohne dass diese ständig zu einer (nicht vorhandenen) Live-Position zurückspringt.

11. Die Menüs im Detail

11.1 Datei

- Flugpfad als GPX exportieren... / als KML exportieren... - speichert alle bisher aufgezeichneten GPS-Punkte des aktuellen Fluges (alternativ das Tracking-Overlay, siehe Abschnitt 8, mit zusätzlicher CSV-Option).
- Beenden

11.2 Route

- Wegpunkt-Modus - schaltet den Klick-zum-Hinzufügen-Modus auf der Karte ein. Ein Klick auf einen bestehenden Wegpunkt entfernt genau diesen wieder.
- Letzten Wegpunkt entfernen / Route löschen
- Wegpunkt-Editor anzeigen - blendet das Editor-Overlay ein/aus (siehe Abschnitt 7.2).
- Route importieren... / Route exportieren... - siehe Abschnitt 7.1.

11.3 Karte

- Kartentyp -> OpenStreetMap / Satellit (Esri).
- Sperrzonen laden... / Sperrzonen anzeigen - siehe Abschnitt 6.2.

11.4 Fluglog

- Log-Einstellungen... - wählt, welche der über 20 Telemetriefelder aufgezeichnet werden sollen und in welchem Intervall (0,1 bis 60 Sekunden).
- Logging aktiv - fragt bei Aktivierung einen Zielpfad für die CSV-Datei ab und beginnt sofort mit der Aufzeichnung; ein erneuter Klick beendet sie.

11.5 Einstellungen

- Verbindung... - wechselt zur Laufzeit zwischen WiFi/UDP und USB/Seriell sowie zwischen MAVLink und CRSF. Beendet dabei automatisch einen laufenden Demo-Modus.
- Akkuwarnung... - siehe Abschnitt 12.
- Home-Position... - siehe Abschnitt 6.5.
- Dashboard anpassen... - siehe Abschnitt 5.2.
- Ansicht -> Auto-Center, Aktuelle Position anspringen (Strg+Pos1), Fahrzeugtyp, Wegpunkt-Editor anzeigen, Koordinaten unter Mauszeiger anzeigen, Künstlicher Horizont anzeigen/Position/Größe.
- Sprache -> Deutsch/English, wechselt die komplette Oberfläche sofort ohne Neustart.

11.6 Simulation

- Demo-Modus - schaltet zur Laufzeit zwischen echter Telemetrie und simulierten Daten um.
- Plan-Modus - siehe Abschnitt 10.

12. Akkuwarnung: LiPo vs. Li-Ion

Sobald der Akku niedrig oder kritisch niedrig wird, gibt die App eine Sprachwarnung aus (offline, über die Windows-SAPI5-Sprachausgabe). Da sich die sichere Entladeschlussspannung von LiPo- und Li-Ion-Zellen deutlich unterscheidet, lässt sich die Chemie unter Einstellungen -> Akkuwarnung... auswählen:

Chemie	Warnung (V/Zelle)	Kritisch (V/Zelle)
LiPo	3,6 V	3,5 V
Li-Ion	3,3 V	3,0 V

Die Auswahl der Chemie füllt automatisch passende Standardwerte vor; Zellenzahl sowie die genauen Warn-/Kritisch-Spannungen lassen sich zusätzlich frei einstellen. Steht eine echte Zellspannungsmessung zur Verfügung (CRSF-Cells-Frame oder MAVLink BATTERY_STATUS), verwendet die App die tatsächliche niedrigste Zellspannung für die Warnung - das ist präziser als eine aus der Gesamtspannung geschätzte Durchschnittszellspannung, da eine einzelne schwache Zelle die tatsächliche Belastungsgrenze bestimmt.

13. Als eigenständige .exe kompilieren

```
cd elrs_ground_station
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt pyinstaller
pyinstaller --name ELRS_GroundStation --onedir main.py
```

Das Ergebnis liegt unter dist\ELRS_GroundStation\ELRS_GroundStation.exe. Der gesamte Ordner dist\ELRS_GroundStation (Exe plus _internal-Verzeichnis mit den Qt/WebEngine-Ressourcen, ca. 500 MB) muss zusammen weitergegeben werden, nicht nur die .exe-Datei allein.

--onedir statt --onefile wird bewusst verwendet: QtWebEngine braucht einen eigenen Hilfsprozess samt Ressourcendateien, die eine Single-File-Exe bei jedem Start erst in ein Temp-Verzeichnis entpacken müsste - das funktioniert, ist aber langsamer beim Start und fehleranfälliger.

Die Exe behält bewusst die Konsole (kein --windowed), damit --list-ports, --demo usw. weiterhin normal über die Kommandozeile nutzbar sind; beim Doppelklick öffnet sich zusätzlich ein Konsolenfenster im Hintergrund.

14. Anhang: Kommandozeilen-Referenz

Option	Beschreibung
--demo	Im Simulationsmodus starten, keine Hardware nötig.
--protocol {mavlink,crsf}	Telemetrieprotokoll (Standard: mavlink).
--connection {udp,usb}	Transportweg (Standard: udp).
--host	Bind-Adresse für UDP-Empfang (Standard: 0.0.0.0).
--port	UDP-Port (Standard: 14550 MAVLink / 14551 CRSF).
--udp-mode {listen,connect}	listen wartet auf Pakete, connect verbindet aktiv zu Host:Port.
--serial-port	USB/seriell-Port bei --connection usb, z. B. COM5.
--baud	Baudrate bei USB (Standard: 57600 MAVLink / 420000 CRSF).
--list-ports	Verfügbare USB/seriell-Ports auflisten und beenden.
--cells	Anzahl LiPo/Li-Ion-Zellen für die Akku-Warnschwellen.
--low-cell-voltage	Zellspannung für die "niedrig"-Warnung.
--critical-cell-voltage	Zellspannung für die "kritisch"-Warnung.
--demo-center lat,lon	Mittelpunkt der Demo-Flugbahn.
--lang {de,en}	Startsprache der Oberfläche.

Vollständige Projektdokumentation, Quellcode und Architektur-Details:

github.com/KresserSimon/ELRS_Telemetry_Groundcontrol